Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Отчет о лабораторной работе № 4

по курсу «Операционные системы»

«Операции с виртуальной памятью»

Студент: Лавров Р. Д.

Группа: ИУ9-42Б

Преподаватель: Брагин А. В.

Москва 2026

1 Содержание

1. Постоновка задачи

2. Практическая реализация

3. Результаты

4. Выводы

5. Список литературы

2 Постоновка задачи

1 часть: Используя созданный в лабораторной работе № 2 минимальный драйвер, совместимый с ОС Windows NT/ReactOS и NetBSD, реализовать следующие действия с виртуальной памятью.

1. Зарезервировать 10 страниц виртуальной памяти, используя функцию ZwAllocateVirtualMemory(NtCurrentProcess, MEM_RESERVE, . . .)..

2. Обеспечить пять первых страниц из выделенных 10 физическими страницами памяти, используя ZwAllocateVirtualMemory(NtCurrentProcess, MEM_COMMIT, . . .).

3. Вывести физические адреса и значения PTE для всех 10 страниц в шестнадцатеричном формате.

4. Освободить выделенную память.Справочную информацию по реализации можно дополнитель-но посмотреть в бюллетене, доступном для скачивания по ссылке https://doc.lagout.org/operating%20system%20/Windows/Memory_Scanning_Under_Wi

2 часть: Используя созданный в лабораторной работе № 2 драйвер, реализовать следующие действия с виртуальной памятью.

1. Зарезервировать 10 страниц виртуальной памяти, используя функцию ядра uvm_km_alloc.

2. Обеспечить пять первых страниц из выделенных 10 физическими страницами памяти, используя функции ядра uvm_pglistalloc и pmap_kenter_pa.

3. Вывести физические адреса и значения PTE для всех 10 страниц в шестнадцатеричном формате.

4. Освободить выделенную память, используя функции ядра pmap_kremove, pmap_update, uvm_pglistfree, uvm_km_free.Драйвер должен выгружаться динамически с помощью команды modunload

3 Практическая реализация

3.1 ReactOS

Первым шагом в файл drivers/CMakelists.txt была внесена строка:

```
add_subdirectory(lab4)
```

После этого в каталоге drivers создана папка lab4, содержащая файлы lab4.c, lab4.rc и CMakeLists.txt.

Листинг 1: CMakeLists.txt

```
add_library(lab4 MODULE
    lab4.c
    lab4.rc
)
set_module_type(lab4 kernelmodedriver)
add_importlibs(lab4 ntoskrnl hal)
add_cd_file(TARGET lab4 DESTINATION reactos/system32/drivers FOR all)
```

Листинг 2: lab4.rc

```
#define REACTOS_VERSION_DLL
#define REACTOS_STR_FILE_DESCRIPTION  "Lab4 Driver"
#define REACTOS_STR_INTERNAL_NAME     "lab4"
#define REACTOS_STR_ORIGINAL_FILENAME "lab4.sys"
#include <reactos/version.rc>
```

Листинг 3: lab4.c

```
#include <ntddk.h>
#include <ntifs.h>
#include <debug.h>

```

```
#define PTE_BASE 0xC0000000
#define MmGetPteAddress(Va) (PVOID)(PTE_BASE + ((((ULONG_PTR)(Va)) >> 12) << 2)
    )

PDEVICE_OBJECT gDeviceObject = NULL;

#define VALID_BIT 0x1
#define DIRTY_BIT 0x40
#define PHYSICAL_ADDRESS_FROM_PTE(pteValue) ((pteValue) & ~0xFFF)

VOID NTAPI DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
    UNREFERENCED_PARAMETER(DriverObject);

    if (gDeviceObject)
    {
        IoDeleteDevice(gDeviceObject);
        gDeviceObject = NULL;
    }

    DPRINT1("Driver unloaded\n");
}

NTSTATUS PerformMemoryOperations(void)
{
    NTSTATUS status;
    PVOID baseAddress = NULL;
    SIZE_T regionSize = 10 * PAGE_SIZE;
    SIZE_T commitSize = 5 * PAGE_SIZE;

    status = ZwAllocateVirtualMemory(NtCurrentProcess(),
                                     &baseAddress,
                                     0,
                                     &regionSize,
                                     MEM_RESERVE,
                                     PAGE_NOACCESS);
    if (!NT_SUCCESS(status))
    {
        DPRINT1("ZwAllocateVirtualMemory (RESERVE) failed: 0x%X\n", status);
        return status;
    }
    DPRINT1("Reserved 10 pages at %p\n", baseAddress);

    status = ZwAllocateVirtualMemory(NtCurrentProcess(),
                                     &baseAddress,
                                     0,
```

```
                                    &commitSize,
                                    MEM_COMMIT,
                                    PAGE_READWRITE);
    if (!NT_SUCCESS(status))
    {
        DPRINT1("ZwAllocateVirtualMemory (COMMIT) failed: 0x%X\n", status);
        ZwFreeVirtualMemory(NtCurrentProcess(), &baseAddress, &regionSize,
    MEM_RELEASE);
        return status;
    }
    DPRINT1("Committed first 5 pages at %p\n", baseAddress);

    for (int i = 0; i < 5; i++)
    {
        PVOID addr = (PVOID)((ULONG_PTR)baseAddress + i * PAGE_SIZE);
        *( volatile UCHAR*)addr = 0xAA;
    }

    for (int i = 0; i < 10; i++)
    {
        PVOID virtualAddr = (PVOID)((ULONG_PTR)baseAddress + i * PAGE_SIZE);
        PVOID ptePtr = MmGetPteAddress(virtualAddr);
        ULONG_PTR pteValue = (ptePtr != NULL) ? *(PULONG_PTR)ptePtr : 0;

        BOOLEAN valid = (pteValue & VALID_BIT) ? TRUE : FALSE;
        ULONG64 physicalAddress = 0;
        if (valid)
        {
            physicalAddress = PHYSICAL_ADDRESS_FROM_PTE(pteValue);
        }

        DPRINT1("Page - %d\n", i);
        DPRINT1("Virtual address - %p\n", virtualAddr);
        DPRINT1("Physical address - %llx\n", physicalAddress);
        DPRINT1("PTE value - %llx\n", (ULONG64)pteValue);
        DPRINT1("Valid - %s\n", valid ? "true" : "false");
        DPRINT1("Dirty - %s\n", (pteValue & DIRTY_BIT) ? "true" : "false");
    }

    regionSize = 0;
    status = ZwFreeVirtualMemory(NtCurrentProcess(),
                                 &baseAddress,
                                 &regionSize,
                                 MEM_RELEASE);
    if (!NT_SUCCESS(status))
    {
```

```
95          DPRINT1("ZwFreeVirtualMemory failed: 0x%X\n", status);
96      }
97      else
98      {
99          DPRINT1("Memory freed successfully\n");
100     }
101
102     return STATUS_SUCCESS;
103 }
104
105 NTSTATUS NTAPI DriverEntry(
106     IN PDRIVER_OBJECT DriverObject,
107     IN PUNICODE_STRING RegistryPath)
108 {
109     UNREFERENCED_PARAMETER(RegistryPath);
110
111     DriverObject->DriverUnload = DriverUnload;
112
113     DPRINT1("Driver Lavrov\n");
114
115     UNICODE_STRING devName = RTL_CONSTANT_STRING(L"\\Device\\lab4dev");
116     NTSTATUS status = IoCreateDevice(
117         DriverObject,
118         0,
119         &devName,
120         FILE_DEVICE_UNKNOWN,
121         0,
122         FALSE,
123         &gDeviceObject
124     );
125
126     if (!NT_SUCCESS(status))
127     {
128         DPRINT1("Failed to create device object: 0x%X\n", status);
129         return status;
130     }
131
132     PerformMemoryOperations();
133
134     return STATUS_SUCCESS;
135 }
```

С помощью команды ninja bootcd был собран ISO-образ, который затем был установлен в QEMU. После этого в ReactOS открыта консоль, и драйвер загружен следующими командами:

```
sc create lab4driver binPath= C:\ReactOS\system32\drivers\ lab4.sys type= kernel
sc start lab4driver
```

3.2 NetBSD

Для реализации драйвера в NetBSD создан файл lab4.c в каталоге usr/src/sys/dev.

Листинг 4: lab4.c

```
#include <sys/module.h>
#include <sys/systm.h>
#include <sys/param.h>
#include <sys/queue.h>
#include <uvm/uvm.h>
#include <machine/pmap.h>

MODULE(MODULE_CLASS_MISC, lab4, NULL);

#define NPAGES 10

static vaddr_t kva;
static struct pglist pglist;

static int
lab4_modcmd(modcmd_t cmd, void *arg)
{
    int i, error;
    struct vm_page *pg;

    switch (cmd) {
    case MODULE_CMD_INIT:
        kva = uvm_km_alloc(kernel_map, NPAGES * PAGE_SIZE, 0, UVM_KMF_VAONLY
    | UVM_KMF
        if (kva == 0)
            return ENOMEM;

        TAILQ_INIT(&pglist);
        error = uvm_pglistalloc(5 * PAGE_SIZE, 0, (paddr_t)-1, PAGE_SIZE, 0, &pglist,
        if (error) {
            uvm_km_free(kernel_map, kva, NPAGES * PAGE_SIZE, UVM_KMF_VAONLY);
            return error;
        }

```

```
34          i = 0;
35          TAILQ_FOREACH(pg, &pglist, pageq.queue) {
36              if (i >= 5) break;
37              pmap_kenter_pa(kva + i * PAGE_SIZE, VM_PAGE_TO_PHYS(pg),
    VM_PROT_READ | V)
38              i++;
39          }
40      pmap_update(pmap_kernel());
41
42          for (i = 0; i < NPAGES; i++) {
43              vaddr_t va = kva + i * PAGE_SIZE;
44              paddr_t pa = 0;
45              if (i < 5)
46                  pmap_extract(pmap_kernel(), va, &pa);
47              printf("Page - %d\n", i);
48              printf("Virtual address - %lx\n", (unsigned long)va);
49              printf("Physical address - %lx\n", (unsigned long)pa);
50          }
51          return 0;
52
53      case MODULE_CMD_FINI:
54          pmap_kremove(kva, NPAGES * PAGE_SIZE);
55          pmap_update(pmap_kernel());
56          uvm_pglistfree(&pglist);
57          uvm_km_free(kernel_map, kva, NPAGES * PAGE_SIZE, UVM_KMF_VAONLY);
58          printf("lab4 unloaded\n");
59          return 0;
60
61      default:
62          return ENOTTY;
63      }
64  }
```

Далее в директории /usr/src/sys/modules была создана папка lab4, в которую добавлен файл Makefile.

Листинг 5: Makefile

```
1  .include "../Makefile.inc"
2  KMOD=lab4
3  .PATH: ${S}/dev
4  SRCS=lab4.c
5  .include <bsd.kmodule.mk>
```

После этого модуль был скомпилирован командой make и загружен с помощью modload lab4.

4 Результаты

4.1 ReactOS

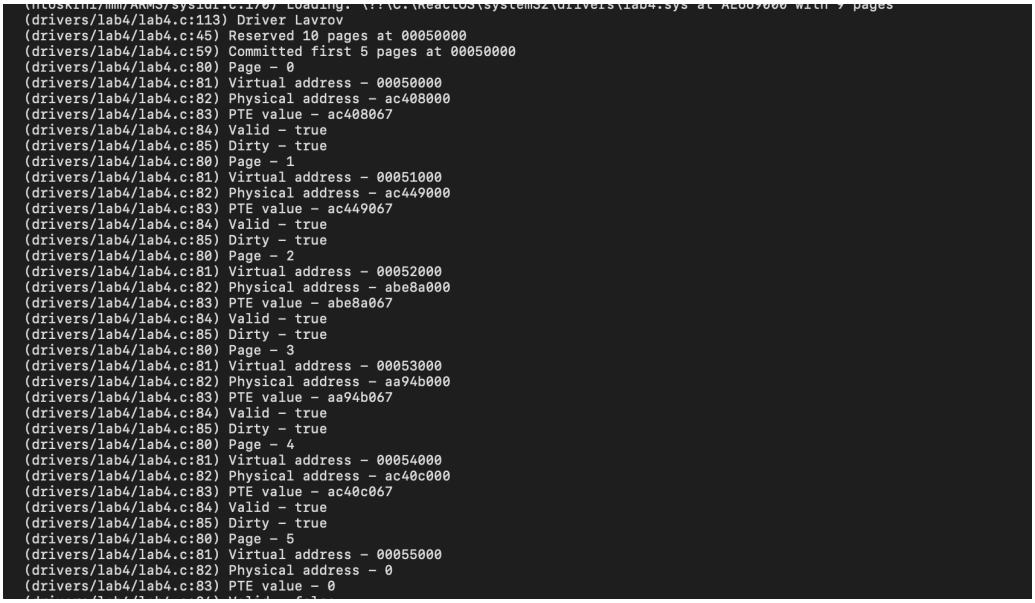


Рис. 1 — Результат работы

4.2 NetBSD

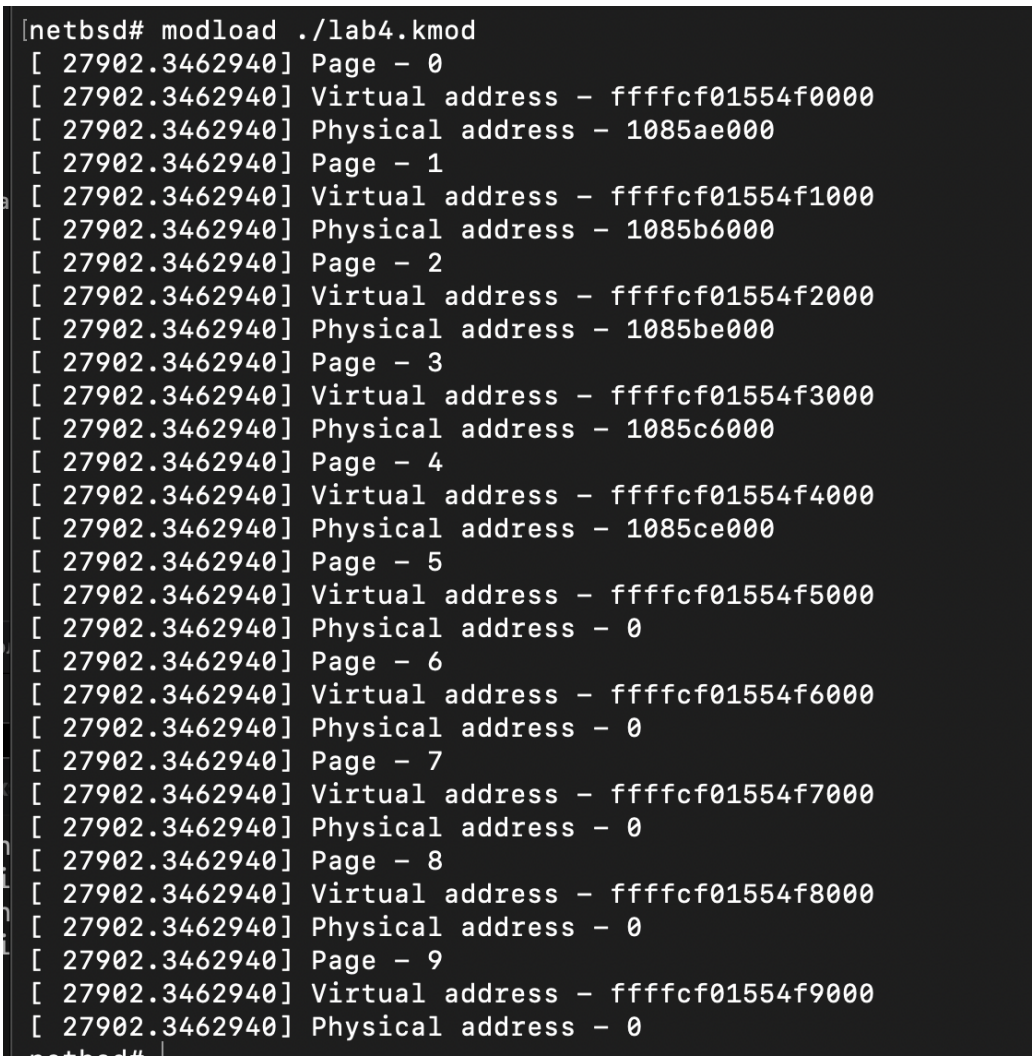


Рис. 2 — Результат работы

5 Выводы

В ходе лабораторной работы реализованы операции с виртуальной памятью в драйверах ReactOS и NetBSD. Для ReactOS с помощью ZwAllocateVirtualMemory выполнены резервирование 10 страниц и выделение физической памяти для первых пяти страниц, выведены физические адреса и значения PTE, память освобождена. Для NetBSD с использованием UVM API (uvm_km_alloc, uvm_pglistalloc, pmap_kenter_pa) проведены аналогичные операции. Корректность работы подтверждена отладочным выводом.

6 Список литературы

1. Учебно-методическое пособие - https://press.bmstu.ru/catalog/item/8226/reader/

2. ReactOS – https://reactos.org

3. ReactOS Build Environment (RosBE) Documentation – https://reactos.org/wiki/Build_Environment

4. NetBSD Project – https://www.netbsd.org

5. The NetBSD Guide – https://www.netbsd.org/docs/guide/en/

6. QEMU Documentation – https://www.qemu.org/docs/master/